

Banco SRJM

Da infraestrutura à aplicação

EKS · Istio · Knative · Vault
Cloudflare · Helm · Argo CD

Você vai aprender	Aplicação no projeto
Caminho da requisição	Do navegador ao PostgreSQL
Integração dos componentes	Quem cria e quem reconcilia cada recurso
Comandos de referência	Instalação, publicação e diagnóstico
Operação e recuperação	Revisões, Secrets, volumes e unseal

Edição de 12 de setembro de 2026

Baseado no ambiente validado em 11/09/2026. O material distingue recursos instalados de artefatos preparados para a migração. Nenhuma credencial foi incluída.

Roteiro de leitura

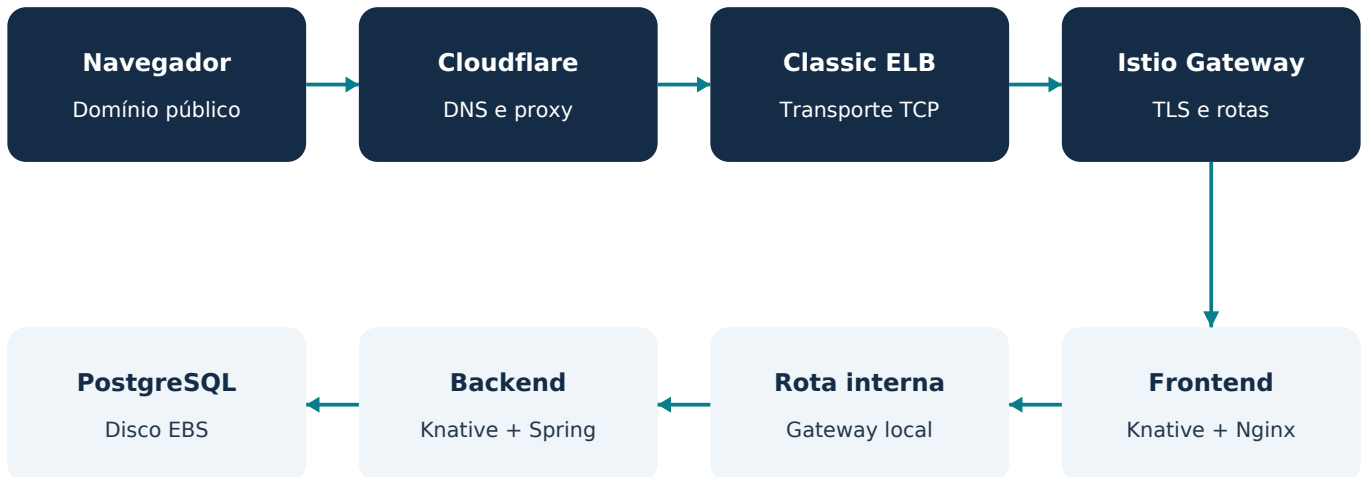
1. Entenda o projeto antes dos comandos	3
2. Kubernetes: o vocabulário que conecta tudo	4
3. EKS, rede AWS, identidade e armazenamento	5
4. AWS Controller e a criação do Classic ELB	6
5. Istio: entrada, listeners, rotas e selectors	7
6. Knative: aplicações, revisões e escala	8
7. Frontend, backend, PostgreSQL e Mailpit	9
8. Cloudflare, cert-manager e HTTPS	10
9. Vault e External Secrets: credenciais sem valores no Git	11
10. YAML, Kustomize, Helm e Argo CD	13
11. Diagnóstico: descubra em qual camada está a falha	15
12. Ordem para reconstruir e mapa dos arquivos	16
Referências e continuidade	18

Os comandos são exemplos de referência para o ambiente descrito. Verifique contexto, dependências e ownership antes de reproduzir uma instalação.

1. Entenda o projeto antes dos comandos

Sua aplicação possui um frontend servido pelo Nginx, um backend Spring Boot e um banco PostgreSQL. Frontend e backend são gerenciados pelo Knative; PostgreSQL executa como StatefulSet. O Mailpit recebe e-mails de teste.

O usuário entra pelo domínio bancosrjm.geradorqr-code-srjm.uk. A Cloudflare encaminha o acesso ao Classic ELB da AWS. O ELB transporta a conexão até o Istio, que usa o roteamento gerado pelo Knative para alcançar a aplicação.



A chamada `/api/` passa pelo frontend antes de alcançar o backend.

A resposta volta pelo caminho de entrada. Fluxo lógico simplificado.

O segundo fluxo distribui credenciais: Vault → External Secrets Operator → Secret Kubernetes → aplicação. A aplicação não consulta o Vault a cada requisição. O cert-manager também consome um Secret sincronizado para validar o domínio na Cloudflare.

O que já estava funcionando: aplicação por manifests/Kustomize, Classic ELB, integração Istio/Knative, certificado e domínio, Vault e sincronização de Secrets. Após uma recriação do pod, o Vault precisou ser desbloqueado para retomar a sincronização.

O que ficou preparado: três charts Helm, pacotes 1.0.0, index.yaml e três Applications do Argo CD. A preparação foi validada por renderização e dry-run; não equivale a uma instalação por Helm/Argo.

Como ler os comandos: alguns reproduzem ajustes efetivamente realizados; outros são comandos equivalentes para reconstrução. Eles não constituem um histórico completo da criação original do EKS. Todos os caminhos locais partem da raiz do repositório.

2. Kubernetes: o vocabulário que conecta tudo

Kubernetes trabalha com estado desejado. Você declara um objeto e um controller observa o cluster para aproximar o estado real dessa declaração. Declarar um Deployment com uma réplica pede ao controller que mantenha uma réplica disponível.

Conceito	Significado	Exemplo do projeto
Cluster	Ambiente Kubernetes completo	srjm-eks-new
Node	Máquina que executa pods	Instâncias EC2 do EKS
Pod	Unidade de execução de containers	Backend e seu queue-proxy
Namespace	Agrupamento lógico de recursos	banco-srjm, vault, istio-ingress
Deployment	Mantém réplicas de uma aplicação	Mailpit e controllers
StatefulSet	Mantém identidade e armazenamento estáveis	PostgreSQL
Service	Endereço de rede estável para pods	postgres e gateway local
ConfigMap	Configuração comum	backend-config
Secret	Dados sensíveis para consumidores autorizados	postgres-secret
PVC / PV	Pedido de armazenamento / volume provisionado	data-postgres-0 e disco EBS
CRD	Novo tipo de recurso registrado na API	ExternalSecret e Knative Serving
Controller / Operator	Reconcilia os objetos que observa	ESO e Knative Operator

Um namespace organiza recursos, mas não configura sozinho isolamento de rede. O acesso depende também de RBAC, políticas e configuração dos componentes.

CRD e controller precisam existir antes do recurso. Um YAML de ExternalSecret não instala o External Secrets Operator. Ele apenas declara uma solicitação que esse controller precisa interpretar.

Também existem dois objetos chamados Service:

API	Responsabilidade
v1, kind Service	Acesso de rede Kubernetes: ClusterIP, NodePort ou LoadBalancer
serving.knative.dev/v1, kind Service	Aplicação Knative: template, revisões, rotas e escala

Um Service Knative gera recursos Kubernetes para executar a aplicação; não deve competir com um Deployment e um Service manuais para o mesmo workload.

3. EKS, rede AWS, identidade e armazenamento

O EKS fornece o plano de controle Kubernetes gerenciado pela AWS. Os nodes executam os pods; a VPC, as subnets, as rotas e os security groups fornecem conectividade. A infraestrutura EKS já existia antes dos ajustes descritos aqui.

Referência	Valor observado
Contexto kubectl	eks-new
Cluster AWS	srjm-eks-new
Região	us-east-2, Ohio
Namespace da aplicação	banco-srjm
Namespace do gateway	istio-ingress
StorageClass	banco-ebs-gp3

O contexto é uma configuração local de acesso, não o nome obrigatório do cluster. Confirme sempre o destino antes de operar:

```
kubectl config current-context
kubectl --context eks-new get nodes
kubectl --context eks-new get namespaces
kubectl --context eks-new get pods -A
helm --kube-context eks-new list -A
```

IAM e RBAC têm papéis diferentes. IAM autoriza chamadas às APIs AWS. RBAC autoriza ações na API Kubernetes. A ServiceAccount do AWS Load Balancer Controller possui IRSA, associando sua identidade Kubernetes a uma role IAM sem guardar access keys nos manifests.

O EBS CSI Driver já estava instalado e provisiona discos para os PVCs. A StorageClass do projeto pede EBS gp3 criptografado, usa WaitForFirstConsumer e política Retain. Assim, o provisionamento considera o agendamento do consumidor; o volume EBS precisa ser compatível com a zona do node.

```
kubectl --context eks-new apply -f k8s/storageclass.yaml
kubectl --context eks-new get storageclass
kubectl --context eks-new -n banco-srjm get pvc
kubectl --context eks-new get pv
```

O PostgreSQL solicita 20 GiB e o Vault 5 GiB. Retain preserva o volume após a liberação correspondente; não substitui backup, replicação ou um plano de restauração.

Argo CD e Karpenter também foram encontrados no cluster. Não foram instalados ou reconfigurados nessa etapa. Escalar pods, tarefa do Knative neste projeto, é diferente de provisionar nodes, função que pode ser exercida por soluções como Karpenter.

4. AWS Controller e a criação do Classic ELB

O AWS Load Balancer Controller foi observado como release Helm em kube-system, chart 3.5.0. Ele gerencia ALB/NLB quando selecionado. **Neste ambiente, quem criou o Classic ELB foi o provedor AWS legado de Services.**

O ajuste decisivo foi desabilitar a mutação que atribui automaticamente a classe NLB aos novos Services LoadBalancer:

```
enableServiceMutatorWebhook: false
```

Ele está em helm/values-aws-load-balancer-controller-classic.yaml. Para preservar a configuração da release existente e aplicar esse ajuste:

```
helm repo add eks https://aws.github.io/eks-charts
helm --kube-context eks-new upgrade aws-load-balancer-controller \
  eks/aws-load-balancer-controller \
  -n kube-system --version 3.5.0 --reuse-values \
  -f helm/values-aws-load-balancer-controller-classic.yaml \
  --wait --timeout 5m
```

Instalação nova: além do chart, prepare a role IAM, sua confiança OIDC e parâmetros de cluster, região, VPC e ServiceAccount. O arquivo acima contém o ajuste do webhook, não toda a infraestrutura de identidade. --reuse-values só faz sentido para uma release existente.

O comportamento legado precisa estar disponível no cluster de destino. Não generalize esse procedimento para todo EKS, especialmente ambientes com seleção de controllers diferente, como Auto Mode. Referência: [AWS Load Balancer Controller e provedor legado](#).

O arquivo k8s/platform/istio-ingress-classic.yaml declara um Service LoadBalancer sem loadBalancerClass e sem anotações NLB. Ele seleciona os pods do Istio e publica TCP 80/443.

```
kubectl --context eks-new apply \
  -f k8s/platform/istio-ingress-classic.yaml
kubectl --context eks-new -n istio-ingress get svc istio-ingress-classic \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

A descoberta automática encontrou duas subnets elegíveis. Subnets públicas precisam de rotas adequadas, identificação e capacidade, além das permissões do controller. Nodes em subnet pública não garantem, sozinhos, o provisionamento.

Hostname observado:

```
ad9260bd1a3f64eeea48e9dcd11746af-1296708679.us-east-2.elb.amazonaws.com
```

Entrada	Destino observado no node	Depois
TCP 80	NodePort 31516	Istio, porta 80
TCP 443	NodePort 31893	Istio, porta 443

Esses números são da instalação observada, não valores universais. O Classic ELB transporta TCP. O TLS da origem termina no Istio, usando o certificado associado ao domínio.

```
aws elb describe-instance-health --region us-east-2 \
  --load-balancer-name ad9260bd1a3f64e44a48e9dcd11746af
```

5. Istio: entrada, listeners, rotas e selectors

O Istio tem três releases no projeto: istio-base, istiod e istio-ingress, todas com charts 1.31.0. A base instala CRDs; o Istiod configura os proxies; o gateway executa os pods Envoy que recebem o tráfego.

Elemento	O que faz
Pod do gateway	Executa o proxy Envoy
Service Kubernetes	Encaminha tráfego aos pods selecionados
Gateway Istio	Configura portas, protocolos e hosts de entrada
VirtualService Istio	Define como rotear as requisições

Esses objetos se complementam. Um recurso Gateway de roteamento não cria, por si só, o Classic ELB do projeto.

Comandos equivalentes de instalação:

```
helm repo add istio https://istio-release.storage.googleapis.com/charts
helm --kube-context eks-new upgrade --install istio-base istio/base \
  -n istio-system --create-namespace --version 1.31.0 --wait
helm --kube-context eks-new upgrade --install istiod istio/istiod \
  -n istio-system --version 1.31.0 \
  -f helm/values-istiod-eks.yaml --wait
helm --kube-context eks-new upgrade --install istio-ingress istio/gateway \
  -n istio-ingress --create-namespace --version 1.31.0 \
  -f helm/values-istio-gateway-eks.yaml --wait
```

O Istiod teve seus requests ajustados para 100m CPU e 512Mi de memória para adequar o consumo reservado à capacidade disponível. O Service do chart gateway ficou em ClusterIP; a entrada pública é o Service Classic separado. Referência: [Istio com Helm](#).

A correção de selectors foi essencial. Os pods do gateway instalado possuem:

```
app: istio-ingress
istio: ingress
```

As referências antigas usavam istio: ingressgateway e não selecionavam os pods corretos. Ajustamos os Gateways gerenciados pelo Knative Serving e seus Services. O Service do gateway local foi colocado em istio-ingress, pois um Service seleciona pods do próprio namespace.

Service	Função
istio-ingress	ClusterIP pertencente ao chart oficial
istio-ingress-classic	Entrada pública pelo Classic ELB
knative-local-gateway	Rotas internas; portas 80 → 8081 e 443 → 8444

Eles usam os mesmos pods, com funções diferentes. A aplicação tem sidecar.istio.io/inject: 'false'. O injector do Istiod continua disponível para o template do gateway. Essa é uma escolha deste perfil; não é uma exigência universal do Knative.

6. Knative: aplicações, revisões e escala

O Knative Operator, chart 1.23.1, reconcilia o recurso KnativeServing. A versão de Serving observada foi 1.23.0. A integração net-istio transforma o roteamento Knative em configuração Istio.

```
helm repo add knative-operator https://knative.github.io/operator
helm --kube-context eks-new upgrade --install knative-operator \
  knative-operator/knative-operator \
  -n knative-operator --create-namespace --version 1.23.1 --wait
kubectl --context eks-new create namespace knative-serving \
  --dry-run=client -o yaml | kubectl --context eks-new apply -f -
kubectl --context eks-new apply -k k8s/platform
kubectl --context eks-new -n knative-serving wait --for=condition=Ready \
  knativeserving/knative-serving --timeout=600s
```

O KnativeServing em k8s/platform/knative-serving.yaml habilita Istio, configura os selectors e aponta para os Services externo e local. O Operator cria/reconcilia os controllers necessários. Referência: [instalação pelo Operator](#).

Recurso Knative	Responsabilidade
Service	Coordena a aplicação
Configuration	Define o template de execução
Revision	Versão imutável do template
Route	Distribui tráfego entre revisões

Frontend e backend usam queue-proxy junto do container da aplicação. Ele participa do encaminhamento, das métricas e da concorrência; não é um sidecar Istio. O Activator pode participar durante ativação e escala, mas não passa obrigatoriamente por toda requisição.

A configuração mantém de 1 a 3 réplicas por revisão ativa. A concorrência máxima configurada por réplica é 80 no frontend e 20 no backend. Esses números não significam usuários ou requisições por segundo. Com mínimo 1, as revisões ativas não escalam a zero por inatividade.

Mudança	Cria revisão?
Apply idêntico	Não
Imagem, variável ou anotação dentro de spec.template	Sim
Apenas distribuição de tráfego	Não; ajusta a rota
Secret externo atualizado	Não automaticamente
Nova imagem enviada à mesma tag latest	Não muda sozinha o template

Os charts calculam checksum do ConfigMap do próprio componente. Para recarregar credenciais externas, altere configVersion nos values. Uma revisão nova só recebe tráfego conforme a configuração da Route; tráfego fixado em revisão antiga não muda automaticamente.

```
kubectl --context eks-new -n banco-srjm get ksvc,revisions
kubectl --context eks-new -n banco-srjm get configurations,routes
kubectl --context eks-new -n banco-srjm describe ksvc backend
```

7. Frontend, backend, PostgreSQL e Mailpit

O Nginx entrega HTML, JavaScript e CSS e encaminha /api/ ao backend. O destino e o Host internos são backend.banco-srjm.svc.cluster.local. O acesso passa pelo roteamento Knative local, preservando a capacidade de distribuir tráfego às revisões.

```
http://backend.banco-srjm.svc.cluster.local:80
```

A porta 80 é a entrada de rede interna; o processo Spring Boot escuta na porta 8080. O backend possui visibilidade Knative cluster-local. O navegador usa o domínio público do frontend e não precisa conhecer o DNS do backend. Esse label define visibilidade da rota; não é uma política completa de isolamento de rede.

O backend consome ConfigMaps e Secrets. O arquivo Spring é montado de backend-application em /etc/banco/application.yaml. O endereço do PostgreSQL é interno, na porta 5432. As tentativas de conexão do Flyway ajudam na inicialização, mas não substituem um banco saudável.

O PostgreSQL usa StatefulSet com uma réplica e PVC de 20 GiB. postgres é o Service de acesso dos clientes; postgres-headless fornece a identidade de rede referenciada pelo StatefulSet. O arquivo foi renomeado para svc-postgres-headless.yaml, preservando o nome Kubernetes e seus selectors.

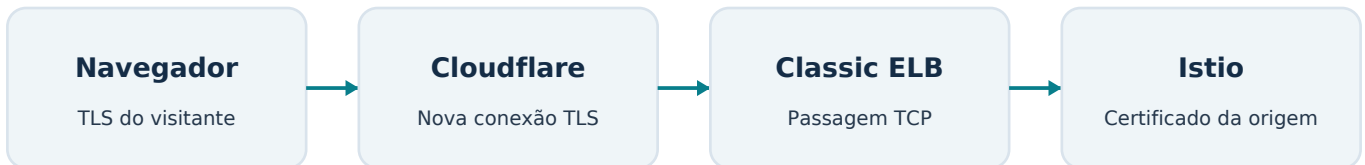
O Mailpit executa como Deployment e captura mensagens SMTP de teste. Ele não entrega e-mails externos e não tem persistência configurada nesse perfil. Ao desabilitá-lo no chart backend, ajuste as configurações e credenciais do SMTP real.

```
kubectl --context eks-new -n banco-srjm get pods,svc,pvc
kubectl --context eks-new -n banco-srjm rollout status \
  statefulset/postgres --timeout=300s
kubectl --context eks-new -n banco-srjm port-forward svc/mailpit 8025:8025
```

Com o port-forward aberto, a interface Mailpit fica em http://localhost:8025.

8. Cloudflare, cert-manager e HTTPS

Na zona geradorqr-code-srjm.uk, o registro CNAME bancosrjm deve apontar para o hostname do Classic ELB. Com proxy habilitado, o navegador conecta à Cloudflare, que estabelece uma conexão separada com a origem.



Full (strict) valida o certificado da origem; o modo do painel não foi reconsultado.

Configure Full (strict) para validar o certificado TLS servido pela origem. O certificado apresentado pela Cloudflare ao visitante é distinto do certificado apresentado pelo Istio à Cloudflare. O teste público com `server: cloudflare` confirmou o proxy, mas não revelou o modo SSL selecionado no painel. Referência: [Full \(strict\)](#).

Dentro do cluster, o tráfego HTTP da aplicação não usa mTLS nesse perfil. Portanto, não descreva esse caminho como criptografado de ponta a ponta entre todos os componentes.

O cert-manager foi observado na versão de chart v1.21.1. Um comando equivalente para instalá-lo e suas CRDs é:

```

helm repo add jetstack https://charts.jetstack.io
helm --kube-context eks-new upgrade --install cert-manager \
  jetstack/cert-manager -n cert-manager --create-namespace \
  --version v1.21.1 --set crds.enabled=true --wait
  
```

Recurso do projeto	Papel
ClusterIssuer letsencrypt-production	Define emissor ACME e validação DNS-01
Certificate banco-srjm	Solicita certificado para o domínio
Secret banco-srjm-tls	Armazena certificado e chave emitidos
ClusterDomainClaim	Reserva domínio para o namespace
DomainMapping	Liga domínio, frontend e Secret TLS

O fluxo DNS-01 é separado das requisições HTTP: cert-manager lê o token Cloudflare, cria um TXT temporário de validação, o Let's Encrypt consulta o DNS e, após a validação, o certificado é emitido e o TXT removido. O cert-manager mantém a renovação. Referência: [solver Cloudflare](#).

O token Cloudflare não encaminha tráfego nem cria o CNAME da aplicação neste projeto. Ele é consumido pelo cert-manager para o desafio DNS. As permissões devem abranger apenas a zona necessária e as operações exigidas pelo solver.

```
kubectl --context eks-new get clusterissuer letsencrypt-production
kubectl --context eks-new -n banco-srjm get certificate, domainmapping
kubectl --context eks-new -n banco-srjm get orders, challenges
curl -I https://bancosrjm.geradorqr-code-srjm.uk/
curl https://bancosrjm.geradorqr-code-srjm.uk/api/actuator/health
```

Abrir apenas o hostname AWS no navegador não testa corretamente o domínio: Host, SNI e certificado precisam corresponder. Para testar a origem diretamente, preservando esses valores:

```
APP_HOST=bancosrjm.geradorqr-code-srjm.uk
ELB_HOST=ad9260bd1a3f64e44a48e9dcd11746af-1296708679.us-east-2.elb.amazonaws.com
curl --no-proxy '*' \
  --connect-to "${APP_HOST}:443:${ELB_HOST}:443" \
  "https://${APP_HOST}/api/actuator/health"
```

9. Vault e External Secrets: credenciais sem valores no Git

O Vault é a origem das credenciais. O External Secrets Operator consulta o Vault usando autenticação Kubernetes e mantém os Secrets consumidos pelos workloads. Configurações comuns permanecem nos ConfigMaps.



As credenciais não estão neste PDF. O app usa o Secret sincronizado.

O ClusterSecretStore descreve conexão, CA e autenticação. O ExternalSecret descreve o caminho remoto e o Secret de destino. Ele não envia automaticamente credenciais existentes para o Vault: essa migração foi feita pelo script de bootstrap.

Store	Restrição de consumidores	Política de leitura
vault-banco	Namespace banco-srjm	Caminhos da aplicação
vault-cert-manager	Namespace cert-manager	Token Cloudflare

Cada store usa uma ServiceAccount e uma política próprias. O termo Cluster significa o escopo do objeto, não acesso irrestrito a todas as credenciais. Os testes de leitura cruzada retornaram 403.

Secret sincronizado	Consumidor
postgres-secret	PostgreSQL e senha de banco do backend
backend-secret	Variáveis sensíveis do backend
backend-application	Arquivo application.yaml do backend

Secret sincronizado	Consumidor
cloudflare-api-token-secret	cert-manager, no namespace cert-manager
frontend-secret	Placeholder preservado, sem uso atual pelo workload
mailpit-secret	Placeholder preservado, sem uso atual pelo workload

A consulta é configurada a cada minuto; disponibilidade, erros e reconciliação podem aumentar o tempo efetivo. Orphan evita vínculo de propriedade de exclusão com o Secret, e Retain conserva o Secret quando os dados remotos são removidos. A aplicação continua dependendo de permissões adequadas sobre os Secrets Kubernetes.

Base64 não é criptografia. O campo data de um Secret usa base64 para representar bytes. Segurança também envolve RBAC, proteção do armazenamento e cuidado com logs, backups e arquivos.

A instalação do Vault usa TLS interno emitido pelo cert-manager, Raft persistente e Service ClusterIP. O injector Vault foi desabilitado porque o ESO já entrega os Secrets. O chart 0.34.1 executava Vault 2.0.4; o chart ESO era 2.10.0.

Pré-requisitos: cert-manager, StorageClass banco-ebs-gp3 e os Secrets iniciais existentes para migração. Estes comandos não geram novas credenciais do nada:

```
helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo add external-secrets https://charts.external-secrets.io
kubectl --context eks-new apply -k k8s/vault
kubectl --context eks-new -n vault wait --for=condition=Ready \
  certificate/vault-ca certificate/vault-server --timeout=180s
helm --kube-context eks-new upgrade --install external-secrets \
  external-secrets/external-secrets \
  -n external-secrets --create-namespace --version 2.10.0 \
  -f helm/values-external-secrets.yaml --wait --timeout 5m
helm --kube-context eks-new upgrade --install vault hashicorp/vault \
  -n vault --version 0.34.1 -f helm/values-vault-eks.yaml --timeout 5m
kubectl --context eks-new -n vault wait \
  --for=jsonpath='{.status.phase}'=Running pod/vault-0 --timeout=300s
python3 scripts/bootstrap-vault.py
kubectl --context eks-new apply -k k8s/external-secrets
```

O bootstrap inicializa uma vez, desbloqueia, habilita KV v2 e autenticação Kubernetes, cria políticas e copia os dados. Depois da sincronização, a finalização verifica valores e isolamento e revoga o token root:

```
kubectl --context eks-new wait --for=condition=Ready \
  clustersecretstore/vault-banco \
  clustersecretstore/vault-cert-manager --timeout=180s
kubectl --context eks-new -n banco-srjm wait --for=condition=Ready \
  externalsecret --all --timeout=180s
kubectl --context eks-new -n cert-manager wait --for=condition=Ready \
  externalsecret/cloudflare-api-token-secret --timeout=180s
python3 scripts/bootstrap-vault.py --verify-and-finalize
```

Recuperação: há uma réplica e unseal manual. Quando o pod foi recriado, o Vault ficou selado, bloqueou consultas e os stores perderam Ready. Os Secrets já sincronizados continuaram disponíveis. Foi necessário executar:

```
python3 scripts/bootstrap-vault.py --unseal
kubectl --context eks-new -n vault get pods,pvc
kubectl --context eks-new get clustersecretstores
kubectl --context eks-new get externalsecrets -A
```

As três partes Shamir estão em `.secrets/vault-init.json`, ignorado pelo Git; são necessárias duas para desbloquear. O arquivo local concentra todas as partes, portanto não representa custódia distribuída. Guarde cópias protegidas e planeje backups de dados/snapshots. Raft com uma réplica não oferece alta disponibilidade.

Para a interface web, gere um token temporário e mantenha o port-forward aberto:

```
python3 scripts/bootstrap-vault.py --admin-token
kubectl --context eks-new -n vault port-forward service/vault 8200:8200
```

Acesse <https://localhost:8200/ui> e escolha Token. O token fica em `.secrets/vault-admin-token`; a CA interna está em `.secrets/vault-ca.crt`. O navegador precisa confiar nessa CA. O token administrativo é limitado aos caminhos configurados, não substitui um token root.

Variáveis de ambiente dos pods existentes não são recarregadas quando o Secret muda. Publique uma revisão que consuma os novos valores. Rotacionar a senha PostgreSQL exige alterar também a senha no banco inicializado.

10. YAML, Kustomize, Helm e Argo CD

Essas ferramentas atuam em etapas diferentes. YAML é o formato dos objetos. Kustomize compõe manifests existentes. Helm renderiza templates usando values e pode gerenciar releases. Argo CD compara o estado desejado com o cluster e executa a sincronização.

Artefato	Uso
k8s/	Alternativa Kustomize preservada, com 36 recursos declarados
helm/charts/	Três charts independentes da aplicação
helm/repository/	Pacotes .tgz e index.yaml
build/helm-rendered/	Recursos renderizados, um por arquivo
argocd/applications/	Três exemplos de Applications com sync manual
helm/values-*.yaml	Ajustes dos charts oficiais da infraestrutura

Cada manifesto/template contém no máximo um recurso. Kustomize usa FIFO para preservar a ordem de envio; isso não espera readiness. Controllers e Secrets devem estar prontos antes de seus consumidores.

Removemos 45 arquivos redundantes da configuração anterior e depois preparamos três charts independentes a pedido do projeto. A pasta k8s/ foi preservada nessa preparação. Manter duas representações no repositório não significa aplicar ambas sobre os mesmos recursos simultaneamente.

Application / chart	Recursos padrão
banco / banco-srjm-banco	PostgreSQL, Services, ConfigMap e ExternalSecret
backend / banco-srjm-backend	API Knative, configuração, dois ExternalSecrets e Mailpit opcional
frontend / banco-srjm-frontend	Frontend Knative, Nginx, Certificate, Claim e DomainMapping

Os charts geram 16 recursos sem sobreposição. A plataforma compartilhada, incluindo stores, Classic ELB e ClusterIssuer, já precisa existir. O store atual só admite banco-srjm; renderizar outro namespace não

concede acesso ao Vault automaticamente.

```
python3 scripts/package-helm.py
```

Esse comando local executa lint, valida referências e paridade, renderiza, empacota, cria o índice e verifica digests. As operações Helm correspondentes são:

```
helm lint helm/charts/banco-srjm-banco \
  helm/charts/banco-srjm-backend \
  helm/charts/banco-srjm-frontend --strict
helm template banco helm/charts/banco-srjm-banco \
  -n banco-srjm --output-dir build/helm-rendered
helm template backend helm/charts/banco-srjm-backend \
  -n banco-srjm --output-dir build/helm-rendered
helm template frontend helm/charts/banco-srjm-frontend \
  -n banco-srjm --output-dir build/helm-rendered
helm package helm/charts/banco-srjm-banco --destination helm/repository
helm package helm/charts/banco-srjm-backend --destination helm/repository
helm package helm/charts/banco-srjm-frontend --destination helm/repository
helm repo index helm/repository
```

O index.yaml usa URLs relativas. Publique-o junto dos três pacotes 1.0.0 no mesmo diretório HTTP(S). Gerar o índice não publica os arquivos. Referência: [repositório de charts Helm](#).

Em um ambiente sem conflito de ownership, com pré-requisitos e credenciais prontos, a instalação direta segue banco → backend → frontend:

```
helm --kube-context eks-new upgrade --install banco \
  helm/repository/banco-srjm-banco-1.0.0.tgz \
  -n banco-srjm --create-namespace --wait --timeout 10m
helm --kube-context eks-new upgrade --install backend \
  helm/repository/banco-srjm-backend-1.0.0.tgz \
  -n banco-srjm --wait --timeout 10m
kubectl --context eks-new -n banco-srjm wait --for=condition=Ready \
  ksvc/backend --timeout=600s
helm --kube-context eks-new upgrade --install frontend \
  helm/repository/banco-srjm-frontend-1.0.0.tgz \
  -n banco-srjm --wait --timeout 10m
kubectl --context eks-new -n banco-srjm wait --for=condition=Ready \
  ksvc/frontend --timeout=600s
```

No cluster observado, os recursos já existem via Kustomize. Helm CLI pode rejeitar a instalação por ownership. Não exclua o banco ou PVC para resolver esse conflito. A migração exige adoção planejada; não foi executada nesta preparação.

O Argo CD já estava instalado. As Applications preparadas usam os charts diretamente no Git e precisam que esses arquivos sejam enviados ao repositório antes da sincronização:

```
kubectl --context eks-new apply -f argocd/applications/banco.yaml
kubectl --context eks-new apply -f argocd/applications/backend.yaml
kubectl --context eks-new apply -f argocd/applications/frontend.yaml
```

Esses comandos cadastram Applications com sync manual. Revise o Diff e sincronize banco, aguarde PostgreSQL/Secret, sincronize backend e depois frontend. Sync-wave ordena recursos dentro de uma Application, não três Applications independentes.

O Argo CD usa Helm para renderizar e gerencia a aplicação dos objetos; não cria uma release em helm list. Depois da adoção, evite atualizações concorrentes pelo Kustomize ou Helm CLI. O primeiro sync pode criar revisões Knative devido às novas anotações de checksum. Referência: [Helm no Argo CD](#).

11. Diagnóstico: descubra em qual camada está a falha

Comece pela observação, não por reinstalar componentes. Verifique estado, condições, eventos e logs. Ready informa a condição publicada pelo controller; Running descreve a fase do pod e não garante que a aplicação esteja pronta.

Camada	Pergunta inicial
DNS / Cloudflare	O domínio resolve e o proxy está ativo conforme esperado?
TLS	O certificado é válido para o domínio e o modo da origem é compatível?
Classic ELB	Existe hostname e os nodes registrados estão InService?
Istio	Os selectors correspondem aos pods e existem endpoints?
Knative	Serviço, revisão e domínio estão Ready?
Frontend / API	Nginx resolve o backend e encaminha o Host correto?
PostgreSQL	Pod pronto, PVC Bound e credenciais correspondentes?
Vault / ESO	Vault desbloqueado, stores Ready e Secrets sincronizados?

```
kubectl --context eks-new -n banco-srjm get pods,svc,pvc
kubectl --context eks-new -n banco-srjm get ksvc,revisions
kubectl --context eks-new -n banco-srjm get certificate,domainmapping
kubectl --context eks-new -n istio-ingress get pods,svc,endpointslices
kubectl --context eks-new get clustersecretstores
kubectl --context eks-new get externalsecrets -A
kubectl --context eks-new -n vault get pods
kubectl --context eks-new -n banco-srjm get events \
  --sort-by=.metadata.creationTimestamp
kubectl --context eks-new -n banco-srjm logs \
  -l serving.knative.dev/service=backend -c backend --tail=100
```

Na validação registrada, o frontend respondeu HTTPS 200, o backend retornou 200 e status UP, DomainMapping/Certificate estavam Ready e os dois nodes do Classic ELB estavam InService. Isso comprova esses testes de disponibilidade; não substitui testes de login e de todas as operações de negócio.

Os testes locais verificam a estrutura sem instalar a aplicação:

```
python3 scripts/validate-manifests.py
python3 scripts/validate-helm.py
kubectl --context eks-new apply --dry-run=server -k k8s
```

Dry-run do servidor valida as requisições contra a API, mas não executa os pods nem comprova disponibilidade futura. Lint, renderização, dry-run e teste HTTP verificam aspectos diferentes.

12. Ordem para reconstruir e mapa dos arquivos

Use as dependências para organizar uma instalação nova. Este repositório não contém uma receita completa de criação de VPC/EKS/IAM do zero; essa base precisa ser provisionada antes.

1. Prepare EKS, rede, nodes, IAM e acesso kubectl.
2. Instale/verifique EBS CSI e a StorageClass.
3. Configure AWS Controller e sua seleção de Services.
4. Instale cert-manager antes de emissores e certificados.
5. Instale Istio base, Istiod e gateway.
6. Instale Knative Operator, crie KnativeServing e espere Ready.
7. Aplique o Service Classic e confirme o balanceador na AWS.
8. Instale Vault e ESO; inicialize ou recupere o Vault conforme o ambiente.
9. Provisione/migre credenciais e espere os Secrets sincronizarem.
10. Suba PostgreSQL, depois backend e frontend.
11. Configure domínio, certificado e CNAME/proxy Cloudflare; teste HTTPS.
12. Adote as Applications no Argo CD e estabeleça rotina de observação e backup.

Pasta / arquivo	Responsabilidade
k8s/backend.yaml e frontend.yaml	Serviços Knative
k8s/postgres.yaml	StatefulSet e template do PVC
k8s/svc-postgres*.yaml	Acesso e identidade de rede do banco
k8s/*-config.yaml e frontend-nginx.yaml	ConfigMaps não sensíveis
k8s/mailpit.yaml e svc-mailpit.yaml	SMTP de testes
k8s/platform/	Classic ELB, gateway local e KnativeServing
k8s/vault/	Namespace, emissores, certificados internos e identidade administrativa
k8s/external-secrets/	Stores, ServiceAccounts, CA pública e sincronizações
k8s/letsencrypt-production.yaml	ClusterIssuer DNS-01
k8s/certificate.yaml	Certificado público
k8s/cluster-domain-claim.yaml e domain-mapping.yaml	Domínio Knative e TLS
scripts/bootstrap-vault.py	Migração, unseal e validação do Vault
scripts/package-helm.py	Pipeline local de validação e empacotamento
helm/charts/ e argocd/applications/	Instalação futura da aplicação por Helm/Argo

Para praticar, acompanhe uma mudança de configuração do chart até o template renderizado, a revisão Knative e a rota. Em seguida, acompanhe uma credencial do caminho Vault ao ExternalSecret, Secret e consumidor. Esses dois exercícios mostram como configuração, reconciliação e execução se relacionam.

As limitações atuais são concretas: PostgreSQL e Vault têm uma réplica, Vault usa unseal manual, persistência não é backup e as imagens usam latest por padrão. Os charts aceitam digest para fixar imagens. Planeje essas evoluções conforme os requisitos de disponibilidade e recuperação da aplicação.

Referências e continuidade

Este PDF foi consolidado a partir das explicações e dos arquivos do projeto, sem consultar novamente o estado vivo do cluster. Nenhum valor de credencial, token ou chave privada foi incluído.

- Repositório: <https://github.com/srjm23/Banco-srjm-k8s>
- Guia técnico local: [docs/arquitetura-e-instalacao.md](#)
- Instalação por Helm/Argo: [helm/README.md](#)
- Operação do Vault: [docs/vault-external-secrets.md](#)
- Ordem e selectors: [docs/estrutura-kustomize.md](#)
- TLS e Cloudflare: [docs/istio-letsencrypt.md](#)
- AWS: <https://docs.aws.amazon.com/eks/latest/userguide/aws-load-balancer-controller.html>
- Istio: <https://istio.io/latest/docs/setup/install/helm/>
- Knative: <https://knative.dev/docs/install/operator/knative-with-operators/>
- ESO/Vault: <https://external-secrets.io/latest/provider/hashicorp-vault/>
- Vault/Helm: <https://developer.hashicorp.com/vault/docs/deploy/kubernetes/helm/configuration>
- Cloudflare: <https://developers.cloudflare.com/ssl/origin-configuration/ssl-modes/full-strict/>
- cert-manager: <https://cert-manager.io/docs/configuration/acme/dns01/cloudflare/>
- Helm: https://helm.sh/docs/topics/chart_repository/
- Argo CD: <https://argo-cd.readthedocs.io/en/stable/user-guide/helm/>